SC2002 Object-Oriented Programming Report
Turn-Based Combat Arena
AY25/26 Sem 2 Group 6 FDAC
Github Repo Link:
https://github.com/HuyTrinh22763/SC2002-Turn-based-Combat

Declaration of Original Work for SC2002/CE2002/CZ2002 Assignment

We hereby declare that the attached group assignment has been researched, undertaken, completed, and submitted as a collective effort by the group members listed below. We have honored the principles of academic integrity and have upheld Student Code of Academic Conduct in the completion of this work. We understand that if plagiarism is found in the assignment, then lower marks or no marks will be awarded for the assessed work. In addition, disciplinary actions may be taken.

NAME	COURSE	LAB	SIGNATURE & DATE
Zhou Yifan	SC2002	FDAC	
Wang Yuehan	SC2002	FDAC	
Venkatesh Mahaa	SC2002	FDAC	
Subasri Vaithiyanathan	SC2002	FDAC	
Trinh Hoang Huy	SC2002	FDAC	

1. DESIGN AND REFLECTION

1.1 OUR DESIGN

1.1.1 Design Consideration:

The battle system is a Java-based console application designed with a focus on modularity, extensibility, and maintainability. It manages turn-based combat between players and enemies, including **action execution, status effects, and item usage**. The system is structured using object-oriented principles to ensure clear separation of responsibilities across different components. This design allows new features, such as additional actions, items, or turn-order rules, to be incorporated with minimal changes to existing code, supporting future scalability and development.

1.1.2 Design Approach:

The overall system is structured into three main layers: the boundary layer, the control layer, and the entity layer. Each layer has a specific role and interacts with the others in a well-defined manner.

The **boundary layer** is responsible for user interaction, including input and output. It is implemented using classes such as `CliGameSession`, `CliInput`, and `CliRenderer`, which handle menu display, user input, and the presentation of battle information. Besides, it also performs basic coordination of the game flow, delegating core battle logic to the control layer.

The **control layer** manages the overall flow of the battle system. The central class, `TurnManager`, coordinates round progression, turn order handling, turn execution, and battle state updates. It interacts with supporting components such as `TurnOrderStrategy` for determining turn order, `PlayerActionValidator` for validating user actions, and `TurnManagerFactory` for constructing the battle system. Factory classes such as `DefaultPlayerFactory` and `DefaultEnemyFactory` are used to create game objects in a structured and decoupled way. The control layer focuses on coordination and delegates detailed action execution to the entity layer.

In the **entity layer**, a specialised battle rules layer defines action-related classes such as `ActionRequest`, `ActionResult`, and `ActionType`, which encapsulate action data and

outcomes. The `ActionProcessor` interface and its implementation `ActionExecutor` handle the execution of actions such as attacks, defence, item usage, and special skills. Level-related classes such as `DifficultyLevel`, `EnemyWave`, `LevelDefinition`, and `LevelCatalog` are used to configure difficulty settings and enemy waves, supporting structured level progression.

The item system is also part of the entity layer. It is designed using the `Item` interface and concrete implementations such as `Potion`, `PowerStone`, and `SmokeBomb`. The `Inventory` class manages item storage and usage, ensuring that item behaviour is encapsulated within each item class. This allows new items to be added easily without modifying existing battle logic.

The interaction flow follows a clear sequence. The user provides input through the boundary layer, which generates an action request. This request is passed to the control layer, where it is validated and coordinated. The control layer then interacts with the entity layer to execute the action, update combatant states, and determine the result. The result is then returned to the boundary layer for display to the user.

This layered design reduces coupling between components and improves maintainability by ensuring clear separation of responsibilities across the boundary, control, and entity layers. By promoting loose coupling and high cohesion, the system is highly flexible, extensible, and easy to maintain.

1.2 HIGHLIGHTS OF SOME DESIGNS

1.2.1 Strategy Pattern for Turn Ordering:

The system applies the Strategy pattern to handle turn ordering through the `TurnOrderStrategy` interface and its implementation `SpeedBasedOrder`. This design separates the turn-order logic from the `TurnManager`, allowing different ordering strategies, such as random or priority-based ordering, to be introduced without modifying the core battle flow. As a result, the system becomes more flexible and adheres to the Open-Closed Principle, since new strategies can be added through additional implementations of the interface.

While this design improves extensibility, it introduces additional abstraction and increases the number of classes involved. For a relatively simple ordering mechanism, this may add some unnecessary complexity, but it provides strong benefits for future scalability.

1.2.2 Centralised Battle Management using TurnManager:

The `TurnManager` class serves as the central controller of the battle system, coordinating round progression, turn execution, battle state updates, and enemy spawning. By centralising the control flow, the system ensures that all battle-related processes follow a consistent structure, making it easier to understand and debug.

At the same time, this design requires careful structuring, as concentrating multiple responsibilities within a single class can lead to increased complexity. To address this, the system delegates specific tasks, such as action execution and validation, to separate components, maintaining a balance between centralisation and modularity.

1.2.3 Separation of Action Processing:

Action handling is separated from battle flow through the use of the `ActionProcessor` interface and its implementation `ActionExecutor`. The `TurnManager` delegates the execution of actions to this component, while action data and results are encapsulated using `ActionRequest` and `ActionResult`. This creates a clear and structured flow for handling actions, improving modularity and making it easier to extend the system with new action types.

1.2.4 Abstraction and Composition of Combatants:

Combatants are designed using the `Combatant` abstraction, supported by the `AbstractCombatant` base class. The `Combatant` interface is composed of multiple smaller interfaces, such as `CombatantStats`, `Damageable`, `CombatStatModifier`, `Stunnable`, `SpecialSkillCooldown`, and `TurnLifecycle`, each representing a specific aspect of combat behaviour. This allows responsibilities to be clearly separated and combined through interface composition.

Concrete classes such as `Player`, `Enemy`, `Warrior`, `Wizard`, `Goblin` and `Wolf` extend this structure to implement specific behaviours. The design also allows future extension of enemy behaviour, as different enemy subclasses can override methods such as `decideAction()` to implement more complex logic. This design supports polymorphism and improves flexibility, as different combatants can be treated uniformly while still maintaining their unique characteristics.

1.2.5 Encapsulation of Item Behaviour:

The item system is designed using the `Item` interface, with concrete implementations such as `Potion`, `PowerStone`, and `SmokeBomb`. Each item encapsulates its own usage logic, validation rules, and effects, while the `Inventory` class manages the storage and usage of items.

This design allows new items to be added with minimal modification to existing code, improving extensibility and maintainability. However, some item behaviours rely on player-specific methods or combatant states, meaning the design is not fully independent. This represents a practical trade-off, as it simplifies implementation while still maintaining a reasonable level of modularity. Overall, the design preserves a clear separation of responsibilities across components.

1.2.6 Factory Design for Object Creation:

Factory classes such as `DefaultPlayerFactory`, `DefaultEnemyFactory`, and `TurnManagerFactory` are used to handle object creation. This separates the instantiation

of objects from the main logic of the system, avoiding tight coupling between components and reducing the need for hardcoded object creation.

This approach improves maintainability and supports future extension, such as introducing new player or enemy types.

1.2.7 Level and Wave Configuration Design:

The system uses classes such as `LevelCatalog`, `LevelDefinition`, `EnemyWave`, and `DifficultyLevel` to represent level configurations and enemy compositions. This separates configuration data from logic, making it easier to manage and extend the game's progression structure.

By representing enemy waves as structured data, the system can support different difficulty levels and backup enemy waves in a flexible manner. Although this introduces additional abstraction, it provides a scalable solution for managing more complex game scenarios.

1.2.8 Validation and Defensive Programming:

Validation mechanisms such as `PlayerActionValidator` and checks within `ActionExecutor` ensure that invalid actions are handled safely. This includes validating targets, item usage, and action conditions before execution.

This improves system robustness and prevents invalid game states, ensuring consistent game behaviour and enhancing the overall reliability of the battle system.

1.3 APPLIED DESIGN PRINCIPLE

1.3.1 Single Responsibility Principle (SRP):

The Single Responsibility Principle recommends that each class should have a clear and singular responsibility, avoiding unrelated tasks. By adhering to SRP, we minimise the ripple effect of changes, simplifying the process of modifying, testing, and reusing code, resulting in more maintainable and robust software design. For example:

Entity Layer

- `AttributeManager` solely handles clamping and calculating combat values (`clampHp`, `calculateDamage`, `isAlive`). If the damage formula changes, only this class needs updating.
- `Inventory` is solely responsible for storing and dispensing items. It does not know what items do or how the player works — it only manages the list.
- `AbstractCombatant` manages the internal state of a combatant (HP, modifiers, stun, cooldown). It does not decide what actions to take or how to display itself.
- `ActionResult` only represents the outcome of an action — messages, damage dealt, and success or failure. `ActionRequest` only carries the inputs needed to execute an action. Neither makes any decisions.
- `SpeedDebuffEffect` is solely responsible for the lifecycle of a speed debuff — attaching to a target, ticking each turn, and expiring. Effect logic is fully isolated from the combatant that hosts it.

Control Layer

- Each action type has its own dedicated executor. `BasicAttackExecutor` handles only damage calculation and smoke bomb interaction. `DefendExecutor` only activates the defend state. `SpecialSkillExecutor` only validates and triggers special skills. `UseItemExecutor` only processes item consumption. `WaitExecutor` only handles the heal-on-wait action. None of these know about each other.
- `ActionExecutor` has one job: routing an `ActionRequest` to the correct executor via a map. It implements no action logic itself.
- `TurnManager` manages battle flow — turn ordering, actor lifecycle, backup wave spawning, and battle state. It delegates all action execution to `ActionProcessor` and all ordering to `TurnOrderStrategy`.
- `PlayerActionValidator` solely validates whether a player's chosen action is legal, keeping validation separate from execution.

- `TurnManagerFactory` only constructs a `TurnManager` from a given difficulty or level number.

Boundary Layer

- `CliInput` only reads and validates raw input from stdin — including exit confirmation. It has no knowledge of game rules or display formatting.
- `CliRenderer` only prints game state to the screen — setup pages, round headers, action results, and the result screen. It never reads input or modifies game state.
- `CliGameSession` only orchestrates the game loop, delegating all rendering to `CliRenderer` and all input to `CliInput`.
- `GameSetup` only holds the player's chosen pre-game configuration (class, difficulty, items). `GameApplication` only contains the `main` entry point.

Each class has a single, well-defined responsibility, making the codebase easier to understand and maintain.

Trade-off: Strict SRP leads to a higher number of small classes. For example, having five separate executor classes (`BasicAttackExecutor`, `DefendExecutor`, `SpecialSkillExecutor`, `UseItemExecutor`, `WaitExecutor`) instead of one combined handler increases the number of files to navigate. However, we accepted this trade-off because the benefit — being able to modify, test, or replace any single action independently without affecting others — outweighs the overhead of additional classes.

1.3.2 Open/Closed Principle (OCP):

The Open/Closed Principle states that classes should be open for extension but closed for modification, allowing for the addition of new functionality without changing existing code. OCP can be implemented through abstraction, inheritance, and polymorphism. For example:

- The `ActionProcessor` interface and `ActionExecutor`'s executor map allow new action types to be added without modifying any existing executor. The `WAIT` action and `WaitExecutor` were introduced this way — only a new class was written and registered in the map. No existing executor was touched.
- The `Item` interface, with `canUse()` and `requiresTarget()` as default methods, allows new item types to be added without modifying existing ones. `Caltrops` was introduced as a new item without any changes to `Potion`, `PowerStone`, or `SmokeBomb`, and `Inventory` and `UseItemExecutor` required zero changes.
- The `TurnObserver` interface allows new status effects to be added without modifying `AbstractCombatant`. Any future effect — poison, burn, regeneration —

is a new class implementing `TurnObserver`. `AbstractCombatant` manages observers through `addStatusEffect()` and never needs modification.

- The `TurnOrderStrategy` interface allows new ordering logic to be plugged in without changing `TurnManager`. Both `SpeedBasedOrder` and `RandomTurnStrategy` exist as interchangeable implementations — adding a third ordering system requires only a new implementing class.
- The `PlayerClass` enum uses abstract methods and anonymous class bodies so that `PlayerClass.WARRIOR.create(name)` and `PlayerClass.WIZARD.create(name)` produce the correct player type. Adding a new class like `Archer` extends the enum without modifying `CliGameSession` or `TurnManagerFactory`.
- `CliRenderer` loops over `PlayerClass.values()` and `EnemyType.values()` dynamically, so new entries in either enum appear in the setup screen automatically — no renderer changes needed.

Trade-off: Designing for OCP requires anticipating the right extension points upfront. Our executor map in `ActionExecutor` works well when new actions follow the same `ActionRequest`/`ActionResult` contract. However, if a fundamentally different kind of action were needed — one that required a completely different request structure — the existing contract would either need modification or workarounds would be required. Anticipating every possible future extension is not always feasible, and over-engineering for extensibility can add unnecessary complexity.

1.3.3 Liskov Substitution Principle (LSP):

To put it simply, the Liskov Substitution Principle states that subtypes must be substitutable for their base types. In this codebase, this principle is followed across all three packages.

In the entity package, all `AbstractCombatant` subclasses including `Enemy`, `Player`, `Goblin`, `Wolf`, `Warrior`, and `Wizard` can be used wherever a `Combatant` is expected.

The `ActionExecutor` in the control package processes basic attacks using only `Combatant` methods such as `takeDamage()`, `isAlive()`, `getAttack()`, and `getDefense()`, never requiring knowledge of the specific subtype.

The `Inventory` system depends entirely on the `Item` interface, allowing `Potion`, `PowerStone`, `SmokeBomb`, and `Caltrops` to be used interchangeably.

The `TurnManager` operates with any `TurnOrderStrategy` implementation, and `SpeedBasedOrder` can be replaced with any future ordering strategy without affecting existing code.

In the boundary package, `CliRenderer` accepts any List of `Combatant` and any `Player` object through their interface methods, and `GameSetup` stores `PlayerClass` and `DifficultyLevel` enums, permitting any enum value to be used interchangeably (shown below).

```
public void renderRoundHeader(int roundNumber, List<Combatant> turnOrder, Player player,
        List<Combatant> enemies) {
```

```
public enum ItemChoice {
    POTION("Potion"),
    POWER_STONE("Power Stone"),
    SMOKE_BOMB("Smoke Bomb"),
    CALTROPS("Caltrops");
```

```
public enum TurnStrategyType {
    SPEED_BASED("Speed-based (Classical)"),
    RANDOM("Random (Saga)");
```

The `ActionResult` factory methods return the same `ActionResult` type for both success and failure paths, so callers in `CliRenderer` need not distinguish between the two paths.

A tradeoff is that `Enemy` declares a `decideAction()` method while `Player` does not. This is acceptable because players and enemies possess fundamentally distinct control mechanisms, with player actions driven by user input collected through `CliInput` in the boundary package and enemies' actions determined by the logic encapsulated within the `Enemy` class hierarchy.

1.3.4 Interface Segregation Principle (ISP):

The Interface Segregation Principle refers to having many specific interfaces rather than one general interface. In other words, we should always avoid designing a 'fat interface'. When developing our system, we noticed that this is important to promote maintainability, flexibility, and modularity. For example:

- Rather than one bloated `Combatant` interface, behaviour is divided into six focused interfaces, each with a single concern:

- `CliRenderer` only calls `CombatantStats` and `Stunnable` methods when rendering — it is never exposed to mutation methods like `takeDamage()` or `modifyAttack()` that are irrelevant to display.
- `ActionProcessor` is a focused interface with a single method `execute(ActionRequest)`, rather than a bloated action interface. Each executor implements exactly this one method with no unrelated obligations.
- `TurnObserver` defines only the four methods a status effect needs — `onAttached`, `onTurnStart`, `onTurnEnd`, and `isExpired`. `SpeedDebuffEffect` implements exactly these without being forced to implement combat stats or item behaviour.
- The `Item` interface uses default methods for `canUse()` and `requiresTarget()` so that items like `Potion` and `SmokeBomb` are never forced to implement target selection logic irrelevant to them.
- `EnemyFactory` and `PlayerFactory` are kept as separate interfaces. `TurnManager` depends only on `EnemyFactory`. `CliGameSession` depends only on `PlayerFactory`. Neither is coupled to factory behaviour it does not need.

Interface	Responsibility
`CombatantStats`	Read-only stats — HP, ATK, DEF, SPD, `isAlive()`
`Damageable`	HP mutation — `takeDamage()`, `heal()`
`CombatStatModifier`	Temporary stat changes — `modifyAttack()`, `modifyDefense()` etc.
`Stunnable`	Stun state — `isStunned()`, `setStunDuration()` etc.
`SpecialSkillCooldown`	Cooldown tracking — `canUseSpecial()`, `reduceCooldown()` etc.
`TurnLifecycle`	Turn hooks — `onTurnStart()`, `onTurnEnd()`

By keeping interfaces lean and focused, we ensure that classes do not implement methods they do not need, reducing coupling and improving maintainability.

Trade-off: Splitting `Combatant` into six interfaces adds indirection — a new developer reading the codebase must look across multiple interface files to understand the full contract of a combatant. When `Combatant` combines all six, it also means that any class accepting a `Combatant` could technically call mutation methods even if it should only be reading stats. A stricter design would pass only `CombatantStats` to `CliRenderer`, but this would require

additional method signatures across the codebase. We accepted this as a manageable trade-off, favouring a pragmatic balance between interface granularity and code simplicity.

1.3.5 Dependency Inversion Principle (DIP):

The Dependency Inversion Principle establishes that high level modules should not depend on low level modules, rather both should depend on abstractions. In this codebase, this principle is followed extensively throughout all three packages.

The `TurnManager` depends on abstractions including `TurnOrderStrategy`, `ActionProcessor`, and `EnemyFactory`, rather than concrete implementations such as `SpeedBasedOrder` or `DefaultEnemyFactory`.

While `TurnManager` receives a concrete `LevelDefinition` object, it relies on the `EnemyFactory` abstraction to transform that definition into `Combatant` instances, avoiding direct instantiation of enemy types.

The `ActionRequest` stores `Combatant` as an abstraction rather than concrete `Player` or `Enemy` types, enabling the same request object to serve any actor. The `Inventory` stores List of `Item` interface rather than concrete item types, permitting new items to be added without modifying `Inventory` code.

The `Player` depends on `Inventory`, which in turn depends on the `Item` interface, establishing a chain of dependencies directed entirely toward abstractions. All action executors including `BasicAttackExecutor`, `DefendExecutor`, `UseItemExecutor`, and `SpecialSkillExecutor` implement the `ActionProcessor` interface, allowing polymorphic usage throughout the system.

The `LevelCatalog` returns List of `LevelDefinition` interface, and `EnemyWave` returns List of `+` interface. Within the boundary package, `CliGameSession` depends on `CliInput` and `CliRenderer`, while `GameSetup` depends on `PlayerClass` and `DifficultyLevel` enums, both of which are stable abstractions.

The `CliRenderer` method `renderResultPage` accepts `BattleState` enum and List of `Combatant` interface rather than concrete implementations.

The `SpeedDebuffEffect` implements the `TurnObserver` interface, permitting attachment to any Combatant without requiring the Combatant to possess specific knowledge of `SpeedDebuffEffect` (shown below).

```java
public class SpeedDebuffEffect implements TurnObserver {

    private final int reductionAmount;
    private int durationRemaining;

    public SpeedDebuffEffect(int reductionAmount, int duration) {
        this.reductionAmount = reductionAmount;
        this.durationRemaining = duration;
    }

    @Override
    public void onAttached(Combatant target) {
        if (target != null) {
            target.modifySpeed(-reductionAmount);
```

This comprehensive adherence to DIP facilitates the addition of new `Combatant` types, new `Item` types, new `ActionType` values, or new `TurnOrderStrategy` implementations with minimal impact on existing systems.

A tradeoff (shown below) is that `TurnManagerFactory` directly instantiates concrete classes including `ActionExecutor` and `DefaultEnemyFactory` rather than receiving them through dependency injection. This design choice is acceptable for a project of this scale, as the factory pattern provides sufficient flexibility for configuration and testing, while implementing a full dependency injection framework would introduce unnecessary complexity.

```java
private static TurnManager fromLevelDefinition(Player player, TurnOrderStrategy turnOrderStrategy,
        LevelDefinition levelDefinition) {
    ActionProcessor actionProcessor = new ActionExecutor();
    EnemyFactory enemyFactory = new DefaultEnemyFactory();
    return new TurnManager(player, null, turnOrderStrategy, actionProcessor, levelDefinition, enemyFactory);
```

Another tradeoff is that `PlayerActionValidator` contains validation logic specific to concrete `ActionType` values including `BASIC_ATTACK, SPECIAL_SKILL, and USE_ITEM`. This is reasonable because `ActionType` represents a stable enumeration with infrequent changes, and centralizing all validation rules within a single validator class improves maintainability by consolidating related logic in one location.

A third tradeoff is that `CliRenderer` directly depends on several concrete entity classes including `Combatant, Player, EnemyType`, and `Item` for display purposes. This is acceptable because rendering logic inherently requires knowledge of the data structures it

presents, and the dependency direction flows from the boundary layer down to the entity layer, never in reverse, thereby preserving clean architecture layering while restricting interactions to read only getter methods that never modify underlying state.

1.4 ADDITIONAL FEATURES

1.4.1 Additional Item: Caltrops

+ ***Reasoning**:* Added to demonstrate how items can manipulate enemy attributes through time-based status effect as well as proving system's robustness against item extensibility
+ ***OODP Implementations:*** Implements the Item interface. Instead of dealing damage, it utilizes `addStatusEffect()` observer channel on the `Combatant` entity. This adheres to the Open-Closed Principle (OCP).
+ ***Technical Proof:***

```
public String use(Player user,
List<Combatant> enemies, Combatant
selectedTarget) {
    for (Combatant enemy : enemies) {
        enemy.addStatusEffect(new
SpeedDebuffEffect(20, 2));
    }
}
```

+ ***Extensibility Proof:*** Adding a "Holy Water" or "Revive Card" item is achieved by creating a new class; no modification to the core `ActionExecutor` is required. The system is open to infinite item variety because it depends on an interface contract.

1.4.2 Additional Action: Wait

+ ***Reasoning:*** Implemented to introduce a defensive-healing strategic option during combat
+ ***OODP Implementation:*** Adheres to the Command Pattern (via `ActionProcessor`) and SRP. The logic for recovery is entirely isolated from the turn management code
+ ***Technical Proof:***

```
int healAmount = (int) (actor.getMaxHp()
* 0.1);
actor.heal(healAmount);
```

+ ***Extensibility Proof:*** Because all actions are registered in a `Map<ActionType, ActionProcessor>`, adding new combat maneuvers like "Guard" or "Cover" only

requires registering a new processor. The combat loop remains Immutable and stable regardless of the number of added actions.

1.4.3 Random Turn Order:

+ ***Reasoning:*** To prove that scheduling calculations can be swapped at runtime to change gameplay dynamics
+ ***OODP Implementation:*** Demonstrates Dependency Inversion (DIP). The `TurnManager` defines the requirement for an order algorithm but does not implement it
+ ***Technical Proof:***

```
// TurnManager depends on the abstraction TurnOrderStrategy
currentTurnOrder =
turnOrderStrategy.determineOrder(getAliveCombatants());
```

+ ***Extensibility Proof:*** If a future requirement asks for “Team-based Rounds” or "Initiative Checks," a developer simply implements the `TurnOrderStrategy` interface. The `TurnManager` remains untouched, proving high robustness against scheduling changes.

1.4.4 Enemy AI Strategy:

+ ***Reasoning:*** To create diverse enemy threats without duplicating combat life cycle code
+ ***OODP Implementation:*** The abstract `Enemy` class manages the turn execution framework, but delegates the choice of action to subclasses via `decideAction()`
+ ***Technical Proof:***

```
public abstract ActionType
decideAction(); // Goblin, Wolf, etc.
implement this
```

+ ***Extensibility Proof:*** Adding a “Dragon Boss” with complex phases is achieved by overriding `decideAction()` in a new subclass. The engine logic in `AbstractCombatant` treats it as any other combatant, which proves Liskov Substitution.

1.4.5 User Early Exiting:

+ ***Reasoning:*** Implementation of a clean termination mechanism from any point in the system.
+ ***OODP Implementation:*** Uses `boundary.QuitGameException` to ripple through the stack. This separates User Intent from the Control Flow Logic
+ ***Extensibility Proof:*** In the future, this can be extended to handle "Player Disconnected" or "System Shutdowns" by simply adding new Exception types and catch blocks, without polling business logic with "quit" boolean flags.

1.4.6 Status Effect Framework:

+ ***Reasoning:*** To allow combatant behavior to be modified dynamically over time
+ ***OODP Implementation:*** Implemented as an observer system. `AbstractCombatant` acts as a host, and `TurnObserver` effects (like `SpeedDebuffEffect`) are composed onto it
+ ***Extensibility Proof:*** Adding "Bleed" or "Confusion" involves zero lines of change to the `Combatant` or `ActionProcessor` classes. Its robustness comes from the fact that new mechanics can be attached to entities at runtime.

1.4.7 Level Catalog:

+ ***Reasoning:*** To separate the content creation (Levels/ Waves) from the engine logic
+ ***OODP Implementation:*** The `LevelCatalog` acts as a blueprint provider. It uses `LevelDefinition` and `EnemyWave` data objects as the source of truth for battle setup
+ ***Extensibility Proof:*** The architecture allows infinite level extension to the game. To expand the game to 100 levels, a designer only needs to add data entries to the `LevelCatalog`. No code changes are required in the `TurnManager`, proving that the system is robust.

1.4.8 TurnManagerFactory:

+ ***Reasoning:*** Assembling a battle (setting up scheduling, wave data and factory) is too complex for a single constructor call
+ ***OODP Implementation:*** High-level abstraction that hides the battle-factors making process
+ ***Extensibility Proof:*** If we add a "Practice Mode" or "Tutorial Mode" in the future, we just add a new factory method. The `TurnManager` class remains stable and focused only on the battle loop
+ The factory also acts as a gating mechanism where we validate level availability before starting a battle.

1.4.9 Attribute Manager: SRP Calculator

+ ***Reasoning:*** HP math, damage mitigation, and clamping rules must be identical across the entire system
+ ***OODP Implementation:*** Single Responsibility Principle(SRP). We moved all the physical attributes of combatants out of the entity to a centralized calculator
+ ***Example:***

```java
// entity.combatants.AttributeManager.java
public static int clampHp(int hp) {
    return Math.max(0, Math.min(999, hp));
}
```

+ ***Extensibility Proof:*** Balancing the game to allow HP up to 99,999 is a one-constant change. Every entity in the game will immediately adhere to the new world-rule.
+ Also, there is no chance that a player's attributes are calculated differently than an enemy's.

1.4.10 Combatant Interface Aggregation:

+ ***Reasoning:*** `Combatant` has too many responsibilities (Stats, Cooldowns, Lifecycle, Damage, Heal, …). One giant interface would not be good for maintenance
+ ***OODP Implementation:*** Interface Segregation Principle(ISP). We defined atomic interfaces like `Damageable`, `Stunnable`, `TurnLifecycle`, … The `Combatant` interface simply aggregates there.
+ ***Technical Proof:***

```
public interface Combatant extends CombatantStats,
Damageable, CombatStatModifier, Stunnable,
SpecialSkillCooldown, TurnLifecycle { ... }
```

+ ***Extensibility Proof:*** If we want to add “Equippable” items, we create a new `Equippable` interface and add it to the aggregation. We do not have to touch the damage logic.
+ Code that only needs to deal damage can interact with `Damageable` interface alone without being forced to implement turn-lifecycle or skill cooldown logic.

1.5 REFLECTION

When we first started this assignment, the combat mechanics seemed very basic. However, we quickly realized that making a game "work" is entirely different from making it follow SOLID principles. Our biggest hardship was the massive "file explosion" that happened as we broke down the system. To keep each class focused on one task (SRP), we ended up with dozens of separate Java files for every small behavior.

This created a major problem for our UML Class Diagram. Trying to fit many small interfaces, like `Stunnable` and `Damageable`, into a single diagram was not easy. The diagram became so crowded with lines and boxes that it was difficult to scale without losing all readability. We spent more time debating how to visualize the system than we did writing the actual code. This was a real trade-off: our code was very clean and flexible, but our overall project structure became much harder to document and oversee.

We managed this difficulty by using the Entity-Control-Boundary (ECB) pattern. This structure gave us a mental map of where everything belonged. Even with a large number of files, we knew that **Boundary classes** only handled the display, **Control classes** handled the logic, and **Entity classes** held the data. This discipline kept the project from becoming a mess of crossed dependencies.

The real proof that this hard work was worth it came when we added features like **Caltrops** and the **WAIT** action. Because we had already separated the system using actions and observers, we could add these features as completely new files. We didn't have to go back and edit the core combat engine at all. This was the moment we realized that the pain of splitting our code was actually a safety net that prevented us from breaking the game when we expanded it.

This project taught us that good software is not just about writing code that runs, but about building a structure that can handle change. We learned that while principles like Interface Segregation and Dependency Inversion make the initial work much harder, they are the only way to keep a project stable as it grows.

For further design and implementation improvements, we suggest the following:

- **Stateful AI Transitions:** Currently, enemies follow a basic action strategy. An improvement would be to implement a "Phase System" where an `Enemy` can swap its `ActionStrategy` object dynamically (e.g., becoming more aggressive when health is below 30%). This would utilize our existing Strategy pattern to create complex boss fights without adding "if-statements" to the core loop.

- **Event-Driven Boundary Updates:** Currently, the UI is updated through direct calls after an action. A more professional approach would be an Event-Listener system. The Combat Engine could publish "Events" (like EntityDamagedEvent), and different **Boundary** observers could then handle the rendering independently. This would completely decouple the timing of logic from the timing of display.
- **Better Package-Level Organization:** As the number of files grew, even our packages started to feel cluttered. Organizing code into more sub-packages would have made the system easier to navigate and document

---

1.6 EXAMPLE DEMO

- *TEST CASE 1:*

iii. Difficulty Level (Medium)

Player: Wizard, **Player Stats:** HP: 200, ATK: 50, DEF: 10, SPD: 20

Items: Power Stone + Potion

Level: Medium - 1 Goblin + 1 Wolf | Backup: Wolf A + Wolf B

Goblin Stats: HP: 55, ATK: 35, DEF: 15, SPD: 25

Wolf Stats: HP: 40, ATK: 45, DEF: 5, SPD: 35

Turn Order: Wolf (SPD 35) → Goblin (SPD 25) → Wizard (SPD 20)

Round 1

Wolf → BasicAttack → Wizard: HP: 200 → 165 (dmg: 45−10=35)

Goblin → BasicAttack → Wizard: HP: 165 → 140 (dmg: 35−10=25)

Wizard → Arcane Blast → All Enemies: Wolf HP: 40 → 0 ✗ ELIMINATED (dmg: 50−5=45) | ATK: 50 → 60 (+10 per Arcane Blast kill) Goblin HP: 55 → 20 (dmg: 50−15=35) | Goblin survives | Cooldown set to 3

End of Round 1: Wizard HP: 140/200 | Goblin HP: 20 | Wolf HP: 0 | Power Stone: 1 | Potion: 1 | Special Skills Cooldown: 3 rounds | Wizard ATK: 60

Round 2

Turn Order: Goblin (SPD 25) → Wizard (SPD 20)

Goblin → BasicAttack → Wizard: HP: 140 → 115 (dmg: 35−10=25)

Wizard → BasicAttack → Goblin: HP: 20 → 0 ✗ ELIMINATED (dmg: 60−15=45) | ATK

Goblin → ELIMINATED: Skipped

All initial enemies eliminated → **Backup Spawn triggered!** Wolf A (HP: 40) + Wolf B (HP: 40) enter simultaneously

End of Round 2: Wizard HP: 115/200 | Goblin HP: 0 | Wolf HP: 0 | Wolf A HP: 40 | Wolf B HP: 40 | Power Stone: 1 | Potion: 1 | Special Skills Cooldown: 2 rounds | Wizard ATK: 60

Round 3

Turn Order: Wolf A (SPD 35) → Wolf B (SPD 35) → Wizard (SPD 20)

Wolf A → BasicAttack → Wizard: HP: 115 → 80 (dmg: 45−10=35)

Wolf B → BasicAttack → Wizard: HP: 80 → 45 (dmg: 45−10=35)

Wizard → Item → Power Stone used → Arcane Blast triggered → All Enemies (ATK: 60): Wolf A HP: 40 → 0 ✗ ELIMINATED (dmg: 60−5=55) | ATK: 60 → 70 (+10) Wolf B HP: 40 → 0 ✗ ELIMINATED (dmg: 70−5=65) | ATK: 70 → 80 (+10) Power Stone consumed | Cooldown unchanged → 2 (Power Stone does not affect cooldown) | All enemies defeated

End of Round 3: Wizard HP: 45/200 | Wolf A HP: 0 | Wolf B HP: 0 | Power Stone: 0 ←- consumed | Potion: 1 ←- unused | Special Skills Cooldown: 2 rounds | Wizard ATK: 80

Victory

Result: Player Victory **Remaining HP:** 45/200 | **Total Rounds:** 3 | **Remaining Power Stone:** 0 | **Remaining Potion:** 1 ←- unused | **Final Wizard ATK:** 80

- *Game Flow:*

```
================================================================================
                          TURN-BASED COMBAT ARENA
================================================================================
                     Enter '0' or 'exit' anytime to quit.
================================================================================
================================================================================
                                  GAME SETUP
================================================================================
PLAYERS:
  - Warrior [HP 260 | ATK 40 | DEF 20 | SPD 30]
  - Wizard [HP 200 | ATK 50 | DEF 10 | SPD 20]

ENEMIES:
  - Goblin [HP 55 | ATK 35 | DEF 15 | SPD 25]
  - Wolf [HP 40 | ATK 45 | DEF 5 | SPD 35]

DIFFICULTY LEVELS:
  - EASY (Level 1) | Initial: 3 Goblin | Backup: None
  - MEDIUM (Level 2) | Initial: 1 Goblin, 1 Wolf | Backup: 2 Wolf
  - HARD (Level 3) | Initial: 2 Goblin | Backup: 1 Goblin, 2 Wolf
================================================================================

[SELECT PLAYER CLASS]
  1) Warrior
  2) Wizard
================================================================================
> 2

[SELECT ITEM FOR SLOT 1]
  1) Potion
  2) Power Stone
  3) Smoke Bomb
  4) Caltrops
================================================================================
> 1

[SELECT ITEM FOR SLOT 2]
  1) Potion
  2) Power Stone
  3) Smoke Bomb
  4) Caltrops
================================================================================
> 2

[SELECT DIFFICULTY]
  1) Easy
  2) Medium
  3) Hard
================================================================================
> 2

[SELECT TURN STRATEGY]
  1) Speed-based (Classical)
  2) Random (Saga)
================================================================================
> 1
================================================================================
```

```
                                    BATTLE START
 ==============================================================================

 [SELECTED SETTINGS]
 - Player Class : Wizard
 - Difficulty   : MEDIUM
 - Turn Strategy: Speed-based (Classical)
 - Item Slot 1  : Potion
 - Item Slot 2  : Power Stone
 ==============================================================================
 Press Enter to continue... (or '0' to exit)

 [ROUND 1]
 TURN ORDER:
   1. Wolf (SPD 35)
   2. Goblin (SPD 25)
   3. Player (SPD 20)

 PLAYER STATUS:
   Player [HP 200/200 | ATK 50 | DEF 10 | SPD 20] ALIVE | CD 0

 ENEMY STATUS:
   - Goblin [HP 55/55 | ATK 35 | DEF 15 | SPD 25] ALIVE
   - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 35] ALIVE
 ==============================================================================

 [TURN EVENT]
 Actor : Wolf
 Type  : Enemy Turn
 Action: BasicAttack
 ==============================================================================

 [ACTION RESULT]
   - Wolf uses BasicAttack on Player for 35 damage.
 ==============================================================================

 [LIVE UPDATE - ROUND 1]
 Player:
   - Player [HP 165/200 | ATK 50 | DEF 10 | SPD 20] ALIVE | CD 0
 Enemies:
   - Goblin [HP 55/55 | ATK 35 | DEF 15 | SPD 25] ALIVE
   - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 35] ALIVE
 ==============================================================================
 Press Enter to continue... (or '0' to exit)

 [TURN EVENT]
 Actor : Goblin
 Type  : Enemy Turn
 Action: BasicAttack
 ==============================================================================

 [ACTION RESULT]
   - Goblin uses BasicAttack on Player for 25 damage.
 ==============================================================================

 [LIVE UPDATE - ROUND 1]
 Player:
   - Player [HP 140/200 | ATK 50 | DEF 10 | SPD 20] ALIVE | CD 0
 Enemies:
   - Goblin [HP 55/55 | ATK 35 | DEF 15 | SPD 25] ALIVE
   - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 35] ALIVE


==============================================================================
Press Enter to continue... (or '0' to exit)

[TURN EVENT]
Actor : Player
Type  : Player Turn
==============================================================================

[ACTION MENU]
  0) Exit Game
  1) BasicAttack
  2) Defend
  3) Item
  4) SpecialSkill
  5) Wait
==============================================================================
> 4

[ACTION RESULT]
  - Player casts ARCANE BLAST (snapshot ATK 50) - 1 kill(s) this cast | total Arcane ATK bonus: +10
==============================================================================
Press Enter to continue... (or '0' to exit)

[END OF ROUND 1 SNAPSHOT]
Player:
  - Player [HP 140/200 | ATK 60 | DEF 10 | SPD 20] ALIVE | CD 3
Enemies:
  - Goblin [HP 20/55 | ATK 35 | DEF 15 | SPD 25] ALIVE
  - Wolf [HP 0/40 | ATK 45 | DEF 5 | SPD 35] ELIMINATED
==============================================================================
Press Enter to continue... (or '0' to exit)

[ROUND 2]
TURN ORDER:
  1. Goblin (SPD 25)
  2. Player (SPD 20)

PLAYER STATUS:
  Player [HP 140/200 | ATK 60 | DEF 10 | SPD 20] ALIVE | CD 2

ENEMY STATUS:
  - Goblin [HP 20/55 | ATK 35 | DEF 15 | SPD 25] ALIVE
==============================================================================

[TURN EVENT]
Actor : Goblin
Type  : Enemy Turn
Action: BasicAttack
==============================================================================
```

```
ENEMY STATUS:
  - Goblin [HP 20/55 | ATK 35 | DEF 15 | SPD 25] ALIVE
==============================================================================

[TURN EVENT]
Actor : Goblin
Type  : Enemy Turn
Action: BasicAttack
==============================================================================

[ACTION RESULT]
  - Goblin uses BasicAttack on Player for 25 damage.
==============================================================================

[LIVE UPDATE - ROUND 2]
Player:
  - Player [HP 115/200 | ATK 60 | DEF 10 | SPD 20] ALIVE | CD 2
Enemies:
  - Goblin [HP 20/55 | ATK 35 | DEF 15 | SPD 25] ALIVE
  - Wolf [HP 0/40 | ATK 45 | DEF 5 | SPD 35] ELIMINATED
==============================================================================
Press Enter to continue... (or '0' to exit)

[TURN EVENT]
Actor : Player
Type  : Player Turn
==============================================================================

[ACTION MENU]
  0) Exit Game
  1) BasicAttack
  2) Defend
  3) Item
  4) SpecialSkill [On Cooldown: 2 round(s)]
  5) Wait
==============================================================================
> 1

[TARGET SELECTION]
  1) Goblin [HP 20/55 | ATK 35 | DEF 15 | SPD 25] ALIVE
==============================================================================
Choose enemy target: 1

[ACTION RESULT]
  - Player uses BasicAttack on Goblin for 45 damage.
  - Goblin is eliminated.
==============================================================================
Press Enter to continue... (or '0' to exit)

[END OF ROUND 2 SNAPSHOT]
Player:
  - Player [HP 115/200 | ATK 60 | DEF 10 | SPD 20] ALIVE | CD 2
Enemies:
  - Goblin [HP 0/55 | ATK 35 | DEF 15 | SPD 25] ELIMINATED
  - Wolf [HP 0/40 | ATK 45 | DEF 5 | SPD 35] ELIMINATED
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 35] ALIVE
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 35] ALIVE
==============================================================================
Press Enter to continue... (or '0' to exit) 


[ROUND 3]
TURN ORDER:
  1. Wolf (SPD 35)
  2. Wolf (SPD 35)
  3. Player (SPD 20)

PLAYER STATUS:
  Player [HP 115/200 | ATK 60 | DEF 10 | SPD 20] ALIVE | CD 1

ENEMY STATUS:
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 35] ALIVE
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 35] ALIVE
==============================================================================

[TURN EVENT]
Actor : Wolf
Type  : Enemy Turn
Action: BasicAttack
==============================================================================

[ACTION RESULT]
  - Wolf uses BasicAttack on Player for 35 damage.
==============================================================================

[LIVE UPDATE - ROUND 3]
Player:
  - Player [HP 80/200 | ATK 60 | DEF 10 | SPD 20] ALIVE | CD 1
Enemies:
  - Goblin [HP 0/55 | ATK 35 | DEF 15 | SPD 25] ELIMINATED
  - Wolf [HP 0/40 | ATK 45 | DEF 5 | SPD 35] ELIMINATED
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 35] ALIVE
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 35] ALIVE
==============================================================================
Press Enter to continue... (or '0' to exit)
```

```
[LIVE UPDATE - ROUND 3]
Player:
  - Player [HP 45/200 | ATK 60 | DEF 10 | SPD 20] ALIVE | CD 1
Enemies:
  - Goblin [HP 0/55 | ATK 35 | DEF 15 | SPD 25] ELIMINATED
  - Wolf [HP 0/40 | ATK 45 | DEF 5 | SPD 35] ELIMINATED
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 35] ALIVE
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 35] ALIVE
==============================================================================
Press Enter to continue... (or '0' to exit)

[TURN EVENT]
Actor : Player
Type  : Player Turn
==============================================================================

[ACTION MENU]
  0) Exit Game
  1) BasicAttack
  2) Defend
  3) Item
  4) SpecialSkill [On Cooldown: 1 round(s)]
  5) Wait
==============================================================================
> 3

[INVENTORY]
  1) Potion
  2) Power Stone
==============================================================================
Choose item to use: 2

[ACTION RESULT]
  - Player casts ARCANE BLAST (snapshot ATK 60) - 2 kill(s) this cast | total Arcane ATK bonus: +30
==============================================================================
Press Enter to continue... (or '0' to exit)

[END OF ROUND 3 SNAPSHOT]
Player:
  - Player [HP 45/200 | ATK 80 | DEF 10 | SPD 20] ALIVE | CD 3
Enemies:
  - Goblin [HP 0/55 | ATK 35 | DEF 15 | SPD 25] ELIMINATED
  - Wolf [HP 0/40 | ATK 45 | DEF 5 | SPD 35] ELIMINATED
  - Wolf [HP 0/40 | ATK 45 | DEF 5 | SPD 35] ELIMINATED
  - Wolf [HP 0/40 | ATK 45 | DEF 5 | SPD 35] ELIMINATED
==============================================================================
Press Enter to continue... (or '0' to exit)
==============================================================================
                            GAME COMPLETION SCREEN
==============================================================================
Congratulations, you have defeated all your enemies.
Statistics: Remaining HP: 45/200 | Total Rounds: 3 | Remaining Potion: 1
 | Remaining Power Stone: 0
==============================================================================

[NEXT STEP]
  1) Replay with same setup
  2) Start a new game
  3) Exit game
==============================================================================
```

- *TESTCASE 2:*

Round 1

Wolf → **Basic Attack** → **Wizard:** HP 200 → 165 (Damage: 45 - 10 = 35)
Goblin → **Basic Attack** → **Wizard:** HP 165 → 140 (Damage: 35 - 10 = 25)
Wizard → **Special Skill (Arcane Blast)** → **All Enemies:**
- **Wolf:** HP 40 → 0 **[X] ELIMINATED** (Damage: 50 - 5 = 45)
- **Goblin:** HP 55 → 20 (Damage: 50 - 15 = 35) | *Survives*
- **Skill Bonus:** 1 kill recorded → Permanent ATK Bonus +10
- **Cooldown:** Set to 3 Rounds

End of Round 1: Wizard HP: 140/200 | Goblin HP: 20 | Wolf HP: 0 | Wizard ATK: **60** | Cooldown: 3

Round 2

Turn Order: Goblin (25) → Wizard (20)

Goblin → **Basic Attack** → **Wizard:** HP 140 → 115 (Damage: 35 - 10 = 25)
Wizard → **Basic Attack** → **Goblin:** HP 20 → 0 **[X] ELIMINATED** (Damage: 60 - 15 = 45)

> [!] **Backup Spawn Triggered!** Wolf A (HP: 40, SPD: 35) and Wolf B (HP: 40, SPD: 35) enter the field.

End of Round 2: Wizard HP: 115/200 | Wolf A HP: 40 | Wolf B HP: 40 | Wizard ATK: **60** | Cooldown: 2

Round 3

Turn Order: Wolf A (35) → Wolf B (35) → Wizard (20)

Wolf A → **Basic Attack** → **Wizard:** HP 115 → 80 (Damage: 45 - 10 = 35)
Wolf B → **Basic Attack** → **Wizard:** HP 80 → 45 (Damage: 45 - 10 = 35)
Wizard → **Item** → **Caltrops used:**
- **Targets:** All Enemies (Wolf A, Wolf B)
- **Effect:** Speed reduced by 20 for 2 turns.
- **New Enemy Speed:** 35 → **15**

End of Round 3: Wizard HP: 45/200 | Wolf A SPD: 15 | Wolf B SPD: 15 | Wizard ATK: **60** | Cooldown: 1

Round 4

Turn Order: Wizard (20) → Wolf A (15) → Wolf B (15)
(Wizard takes the first turn due to Speed Tier advantage!)

Wizard → **Action** → **Wait:**
- **Effect:** Recovers 10% of Max HP.
- **HP Recovery:** 45 → **65** (+20 HP)

Wolf A → **Basic Attack** → **Wizard:**
- **Damage:** 45 - 10 = 35
- **Wizard HP:** 65 → 30

Wolf B → **Basic Attack** → **Wizard:**
- **Damage:** 45 - 10 = 35
- **Wizard HP:** 30 → 0 **[X] ELIMINATED**

Defeat

Final Result: Player Defeated
Total Rounds: 4
Remaining Items: Power Stone: 1 (Unused) | Caltrops: 0 (Consumed)
Final Wizard ATK: 60 (Base 50 + R1 bonus 10)
Status: Strategic failure. Healing via Wait was insufficient to survive the final synchronized assault.

- *Gameflow:*

```
================================================================================
                              TURN-BASED COMBAT ARENA
================================================================================
                       Enter '0' or 'exit' anytime to quit.
================================================================================
================================================================================
                                     GAME SETUP
================================================================================
PLAYERS:
  - Warrior [HP 260 | ATK 40 | DEF 20 | SPD 30]
  - Wizard [HP 200 | ATK 50 | DEF 10 | SPD 20]

ENEMIES:
  - Goblin [HP 55 | ATK 35 | DEF 15 | SPD 25]
  - Wolf [HP 40 | ATK 45 | DEF 5 | SPD 35]

DIFFICULTY LEVELS:
  - EASY (Level 1) | Initial: 3 Goblin | Backup: None
  - MEDIUM (Level 2) | Initial: 1 Goblin, 1 Wolf | Backup: 2 Wolf
  - HARD (Level 3) | Initial: 2 Goblin | Backup: 1 Goblin, 2 Wolf
================================================================================

[SELECT PLAYER CLASS]
  1) Warrior
  2) Wizard
================================================================================
> 2

[SELECT ITEM FOR SLOT 1]
  1) Potion
  2) Power Stone
  3) Smoke Bomb
  4) Caltrops
================================================================================
> 2

[SELECT ITEM FOR SLOT 2]
  1) Potion
  2) Power Stone
  3) Smoke Bomb
  4) Caltrops
================================================================================
> 4

[SELECT DIFFICULTY]
  1) Easy
  2) Medium
  3) Hard
================================================================================
> 2

[SELECT TURN STRATEGY]
  1) Speed-based (Classical)
  2) Random (Saga)
================================================================================
> 1
================================================================================
                                   BATTLE START
================================================================================


[SELECTED SETTINGS]
- Player Class : Wizard
- Difficulty   : MEDIUM
- Turn Strategy: Speed-based (Classical)
- Item Slot 1  : Power Stone
- Item Slot 2  : Caltrops
================================================================================
Press Enter to continue... (or '0' to exit)

[ROUND 1]
TURN ORDER:
  1. Wolf (SPD 35)
  2. Goblin (SPD 25)
  3. Player (SPD 20)

PLAYER STATUS:
  Player [HP 200/200 | ATK 50 | DEF 10 | SPD 20] ALIVE | CD 0

ENEMY STATUS:
  - Goblin [HP 55/55 | ATK 35 | DEF 15 | SPD 25] ALIVE
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 35] ALIVE
================================================================================

[TURN EVENT]
Actor : Wolf
Type  : Enemy Turn
Action: BasicAttack
================================================================================

[ACTION RESULT]
  - Wolf uses BasicAttack on Player for 35 damage.
================================================================================

[LIVE UPDATE - ROUND 1]
Player:
  - Player [HP 165/200 | ATK 50 | DEF 10 | SPD 20] ALIVE | CD 0
Enemies:
  - Goblin [HP 55/55 | ATK 35 | DEF 15 | SPD 25] ALIVE
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 35] ALIVE
================================================================================
Press Enter to continue... (or '0' to exit)

[TURN EVENT]
Actor : Goblin
Type  : Enemy Turn
Action: BasicAttack
================================================================================

[ACTION RESULT]
  - Goblin uses BasicAttack on Player for 25 damage.
================================================================================
```

```
[SELECTED SETTINGS]
- Player Class : Wizard
- Difficulty   : MEDIUM
- Turn Strategy: Speed-based (Classical)
- Item Slot 1  : Power Stone
- Item Slot 2  : Caltrops
==============================================================================
Press Enter to continue... (or '0' to exit)

[ROUND 1]
TURN ORDER:
  1. Wolf (SPD 35)
  2. Goblin (SPD 25)
  3. Player (SPD 20)

PLAYER STATUS:
  Player [HP 200/200 | ATK 50 | DEF 10 | SPD 20] ALIVE | CD 0

ENEMY STATUS:
  - Goblin [HP 55/55 | ATK 35 | DEF 15 | SPD 25] ALIVE
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 35] ALIVE
==============================================================================

[TURN EVENT]
Actor : Wolf
Type  : Enemy Turn
Action: BasicAttack
==============================================================================

[ACTION RESULT]
  - Wolf uses BasicAttack on Player for 35 damage.
==============================================================================

[LIVE UPDATE - ROUND 1]
Player:
  - Player [HP 165/200 | ATK 50 | DEF 10 | SPD 20] ALIVE | CD 0
Enemies:
  - Goblin [HP 55/55 | ATK 35 | DEF 15 | SPD 25] ALIVE
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 35] ALIVE
==============================================================================
Press Enter to continue... (or '0' to exit)

[TURN EVENT]
Actor : Goblin
Type  : Enemy Turn
Action: BasicAttack
==============================================================================

[ACTION RESULT]
  - Goblin uses BasicAttack on Player for 25 damage.
==============================================================================


[LIVE UPDATE - ROUND 1]
Player:
  - Player [HP 140/200 | ATK 50 | DEF 10 | SPD 20] ALIVE | CD 0
Enemies:
  - Goblin [HP 55/55 | ATK 35 | DEF 15 | SPD 25] ALIVE
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 35] ALIVE
==============================================================================
Press Enter to continue... (or '0' to exit)

[TURN EVENT]
Actor : Player
Type  : Player Turn
==============================================================================

[ACTION MENU]
  0) Exit Game
  1) BasicAttack
  2) Defend
  3) Item
  4) SpecialSkill
  5) Wait
==============================================================================
> 4

[ACTION RESULT]
  - Player casts ARCANE BLAST (snapshot ATK 50) - 1 kill(s) this cast | total Arcane ATK bonus: +10
==============================================================================
Press Enter to continue... (or '0' to exit)

[END OF ROUND 1 SNAPSHOT]
Player:
  - Player [HP 140/200 | ATK 60 | DEF 10 | SPD 20] ALIVE | CD 3
Enemies:
  - Goblin [HP 20/55 | ATK 35 | DEF 15 | SPD 25] ALIVE
  - Wolf [HP 0/40 | ATK 45 | DEF 5 | SPD 35] ELIMINATED
==============================================================================
Press Enter to continue... (or '0' to exit)

[ROUND 2]
TURN ORDER:
  1. Goblin (SPD 25)
  2. Player (SPD 20)

PLAYER STATUS:
  Player [HP 140/200 | ATK 60 | DEF 10 | SPD 20] ALIVE | CD 2

ENEMY STATUS:
  - Goblin [HP 20/55 | ATK 35 | DEF 15 | SPD 25] ALIVE
==============================================================================

[TURN EVENT]
Actor : Goblin
Type  : Enemy Turn
Action: BasicAttack
==============================================================================
```

```
[LIVE UPDATE - ROUND 2]
Player:
  - Player [HP 115/200 | ATK 60 | DEF 10 | SPD 20] ALIVE | CD 2
Enemies:
  - Goblin [HP 20/55 | ATK 35 | DEF 15 | SPD 25] ALIVE
  - Wolf [HP 0/40 | ATK 45 | DEF 5 | SPD 35] ELIMINATED
==============================================================================
Press Enter to continue... (or '0' to exit)

[TURN EVENT]
Actor : Player
Type  : Player Turn
==============================================================================

[ACTION MENU]
  0) Exit Game
  1) BasicAttack
  2) Defend
  3) Item
  4) SpecialSkill [On Cooldown: 2 round(s)]
  5) Wait
==============================================================================
>
Invalid input. Please enter a number between 1 and 5.
>
Invalid input. Please enter a number between 1 and 5.
> 1

[TARGET SELECTION]
  1) Goblin [HP 20/55 | ATK 35 | DEF 15 | SPD 25] ALIVE
==============================================================================
Choose enemy target: 1

[ACTION RESULT]
  - Player uses BasicAttack on Goblin for 45 damage.
  - Goblin is eliminated.
==============================================================================
Press Enter to continue... (or '0' to exit)

[END OF ROUND 2 SNAPSHOT]
Player:
  - Player [HP 115/200 | ATK 60 | DEF 10 | SPD 20] ALIVE | CD 2
Enemies:
  - Goblin [HP 0/55 | ATK 35 | DEF 15 | SPD 25] ELIMINATED
  - Wolf [HP 0/40 | ATK 45 | DEF 5 | SPD 35] ELIMINATED
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 35] ALIVE
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 35] ALIVE
==============================================================================
Press Enter to continue... (or '0' to exit)


[ROUND 3]
TURN ORDER:
  1. Wolf (SPD 35)
  2. Wolf (SPD 35)
  3. Player (SPD 20)

PLAYER STATUS:
  Player [HP 115/200 | ATK 60 | DEF 10 | SPD 20] ALIVE | CD 1

ENEMY STATUS:
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 35] ALIVE
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 35] ALIVE
==============================================================================

[TURN EVENT]
Actor : Wolf
Type  : Enemy Turn
Action: BasicAttack
==============================================================================

[ACTION RESULT]
  - Wolf uses BasicAttack on Player for 35 damage.
==============================================================================

[LIVE UPDATE - ROUND 3]
Player:
  - Player [HP 80/200 | ATK 60 | DEF 10 | SPD 20] ALIVE | CD 1
Enemies:
  - Goblin [HP 0/55 | ATK 35 | DEF 15 | SPD 25] ELIMINATED
  - Wolf [HP 0/40 | ATK 45 | DEF 5 | SPD 35] ELIMINATED
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 35] ALIVE
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 35] ALIVE
==============================================================================
Press Enter to continue... (or '0' to exit)

[TURN EVENT]
Actor : Wolf
Type  : Enemy Turn
Action: BasicAttack
==============================================================================

[ACTION RESULT]
  - Wolf uses BasicAttack on Player for 35 damage.
==============================================================================

[LIVE UPDATE - ROUND 3]
Player:
  - Player [HP 45/200 | ATK 60 | DEF 10 | SPD 20] ALIVE | CD 1
Enemies:
  - Goblin [HP 0/55 | ATK 35 | DEF 15 | SPD 25] ELIMINATED
  - Wolf [HP 0/40 | ATK 45 | DEF 5 | SPD 35] ELIMINATED
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 35] ALIVE
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 35] ALIVE
==============================================================================
Press Enter to continue... (or '0' to exit)
```

```
[TURN EVENT]
Actor : Player
Type  : Player Turn
==============================================================================

[ACTION MENU]
  0) Exit Game
  1) BasicAttack
  2) Defend
  3) Item
  4) SpecialSkill [On Cooldown: 1 round(s)]
  5) Wait
==============================================================================
> 3

[INVENTORY]
  1) Power Stone
  2) Caltrops
==============================================================================
Choose item to use: 2

[ACTION RESULT]
  - Player threw Caltrops! All enemies lose 20 speed for 2 turns!
==============================================================================
Press Enter to continue... (or '0' to exit)

[END OF ROUND 3 SNAPSHOT]
Player:
  - Player [HP 45/200 | ATK 60 | DEF 10 | SPD 20] ALIVE | CD 1
Enemies:
  - Goblin [HP 0/55 | ATK 35 | DEF 15 | SPD 25] ELIMINATED
  - Wolf [HP 0/40 | ATK 45 | DEF 5 | SPD 35] ELIMINATED
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 15] ALIVE
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 15] ALIVE
==============================================================================
Press Enter to continue... (or '0' to exit)

[ROUND 4]
TURN ORDER:
  1. Player (SPD 20)
  2. Wolf (SPD 15)
  3. Wolf (SPD 15)

PLAYER STATUS:
  Player [HP 45/200 | ATK 60 | DEF 10 | SPD 20] ALIVE | CD 0

ENEMY STATUS:
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 15] ALIVE
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 15] ALIVE
==============================================================================

[TURN EVENT]
Actor : Player
Type  : Player Turn
==============================================================================

[ACTION MENU]
  0) Exit Game
  1) BasicAttack
  2) Defend
  3) Item
  4) SpecialSkill
  5) Wait
==============================================================================
> 5

[ACTION RESULT]
  - Player uses Wait and recovers 20 HP.
==============================================================================
Press Enter to continue... (or '0' to exit)

[TURN EVENT]
Actor : Wolf
Type  : Enemy Turn
Action: BasicAttack
==============================================================================

[ACTION RESULT]
  - Wolf uses BasicAttack on Player for 35 damage.
==============================================================================

[LIVE UPDATE - ROUND 4]
Player:
  - Player [HP 30/200 | ATK 60 | DEF 10 | SPD 20] ALIVE | CD 0
Enemies:
  - Goblin [HP 0/55 | ATK 35 | DEF 15 | SPD 25] ELIMINATED
  - Wolf [HP 0/40 | ATK 45 | DEF 5 | SPD 35] ELIMINATED
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 15] ALIVE
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 15] ALIVE
==============================================================================
Press Enter to continue... (or '0' to exit)

[TURN EVENT]
Actor : Wolf
Type  : Enemy Turn
Action: BasicAttack
==============================================================================

[ACTION RESULT]
  - Wolf uses BasicAttack on Player for 35 damage.
  - Player is eliminated.
==============================================================================
```

```
[LIVE UPDATE - ROUND 4]
Player:
  - Player [HP 0/200 | ATK 60 | DEF 10 | SPD 20] ELIMINATED | CD 0
Enemies:
  - Goblin [HP 0/55 | ATK 35 | DEF 15 | SPD 25] ELIMINATED
  - Wolf [HP 0/40 | ATK 45 | DEF 5 | SPD 35] ELIMINATED
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 15] ALIVE
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 15] ALIVE
==============================================================================
Press Enter to continue... (or '0' to exit)

[END OF ROUND 4 SNAPSHOT]
Player:
  - Player [HP 0/200 | ATK 60 | DEF 10 | SPD 20] ELIMINATED | CD 0
```

```
[TURN EVENT]
Actor : Wolf
Type  : Enemy Turn
Action: BasicAttack
==============================================================================

[ACTION RESULT]
  - Wolf uses BasicAttack on Player for 35 damage.
  - Player is eliminated.
==============================================================================

[LIVE UPDATE - ROUND 4]
Player:
  - Player [HP 0/200 | ATK 60 | DEF 10 | SPD 20] ELIMINATED | CD 0
Enemies:
  - Goblin [HP 0/55 | ATK 35 | DEF 15 | SPD 25] ELIMINATED
  - Wolf [HP 0/40 | ATK 45 | DEF 5 | SPD 35] ELIMINATED
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 15] ALIVE
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 15] ALIVE
==============================================================================
Press Enter to continue... (or '0' to exit)

[END OF ROUND 4 SNAPSHOT]
Player:
  - Player [HP 0/200 | ATK 60 | DEF 10 | SPD 20] ELIMINATED | CD 0
Enemies:
  - Goblin [HP 0/55 | ATK 35 | DEF 15 | SPD 25] ELIMINATED
  - Wolf [HP 0/40 | ATK 45 | DEF 5 | SPD 35] ELIMINATED
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 15] ALIVE
  - Wolf [HP 40/40 | ATK 45 | DEF 5 | SPD 15] ALIVE
==============================================================================
Press Enter to continue... (or '0' to exit)
==============================================================================
                              GAME COMPLETION SCREEN
==============================================================================
Defeated. Don't give up, try again!
Statistics: Enemies remaining: 2 | Total Rounds Survived: 4
 | Remaining Power Stone: 1
 | Remaining Caltrops: 0
==============================================================================

[NEXT STEP]
  1) Replay with same setup
  2) Start a new game
  3) Exit game
==============================================================================
>
```

2. <u>IMAGES AND DIAGRAMS</u>

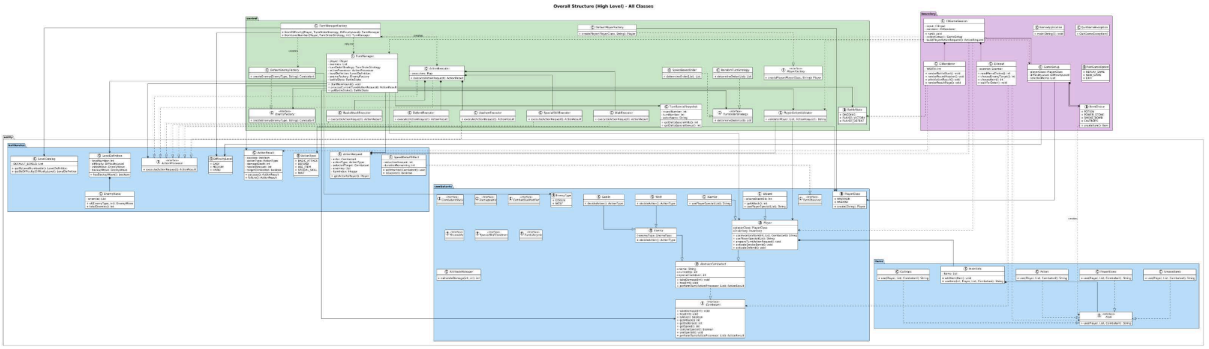
2.1 FILE NAMES INSIDE ZIP FOLDER

- **Report:**
  Github: "/FDAC_Grp6_Report.pdf**"**
- **UML whole class diagram:**
  Github: "/images/UML class diagram - Whole_Codebase.pdf"
- **UML class diagram for Entity Package:**
  Github: "/images/UML class diagram - Entity_Package.pdf"
- **UML class diagram for Control Package:**
  Github: "/images/UML class diagram - Control_Package"
- **UML class diagram for Boundary Package:**
  Github: "/images/UML class diagram - Boundary_Package"
- **UML sequence diagram for "Wolf uses Basic Attack and eliminate Warrior":**
  Github: "/images/wolf_warrior_defeat_sequence.png"
- **UML sequence diagram for "Warrior does Shield Bash on Goblin, stunning it":**
  Github: "/images/shield_bash_sequence.png"
- **UML sequence diagram for "Warrior executes basic attack, eliminating Goblin":**
  Github: "/images/basic_attack_elimination_sequence.png"

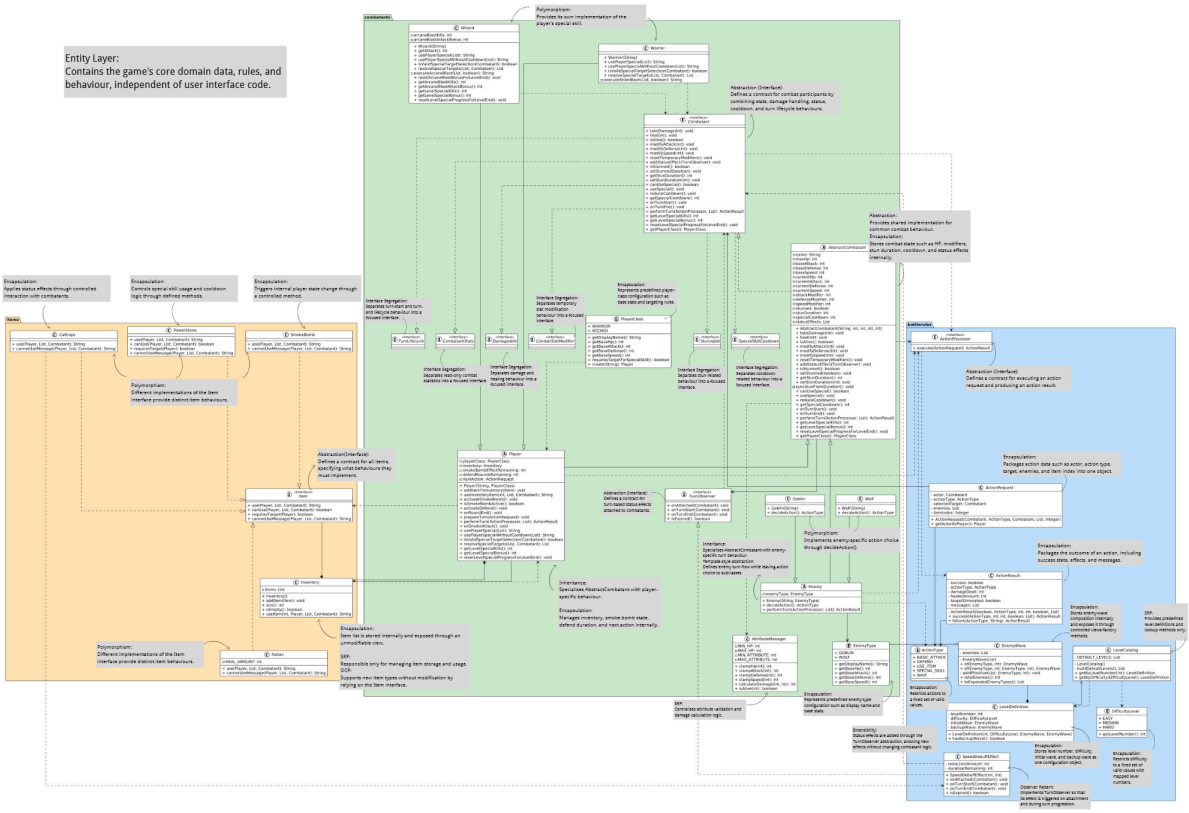
<u>Notes:</u> We decided to not annotate on the Whole Codebase Class Diagram on its own, but instead choose to annotate on each individual package in the codebase namely: Entity, Control and Boundary. We chose this approach because we believed that the Whole Codebase Class Diagram has already been too complicated and annotations would not be as clear as those on individual packages. We have made sure that every important implementation of SOLID principles as well as OOP concepts like Inheritance or Polymorphism have been noted in the 3 individual UML class diagrams for each package, which aligns to the assignment's requirements on diagram annotations.

2.2 APPENDIX

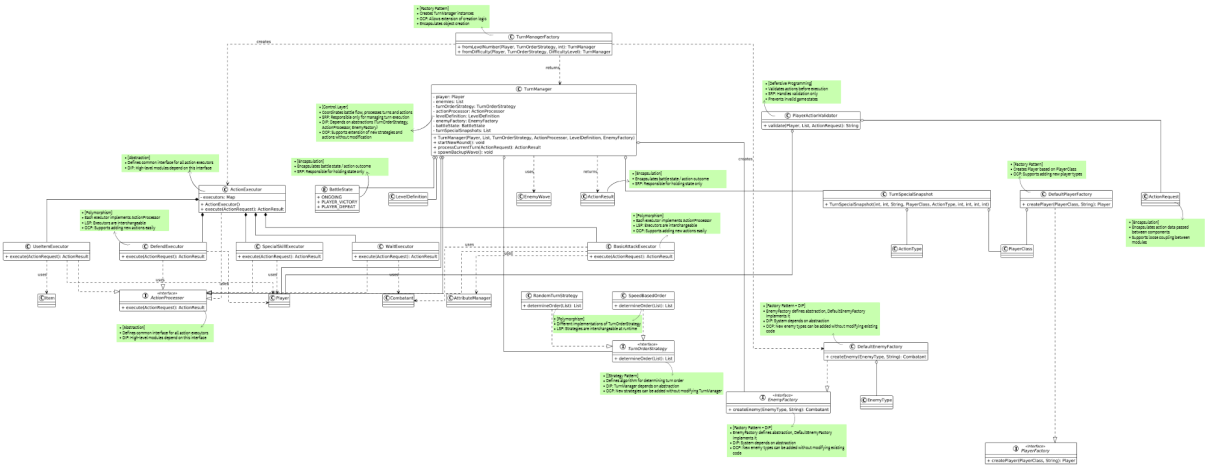
2.2.1: Whole Codebase UML Class Diagram



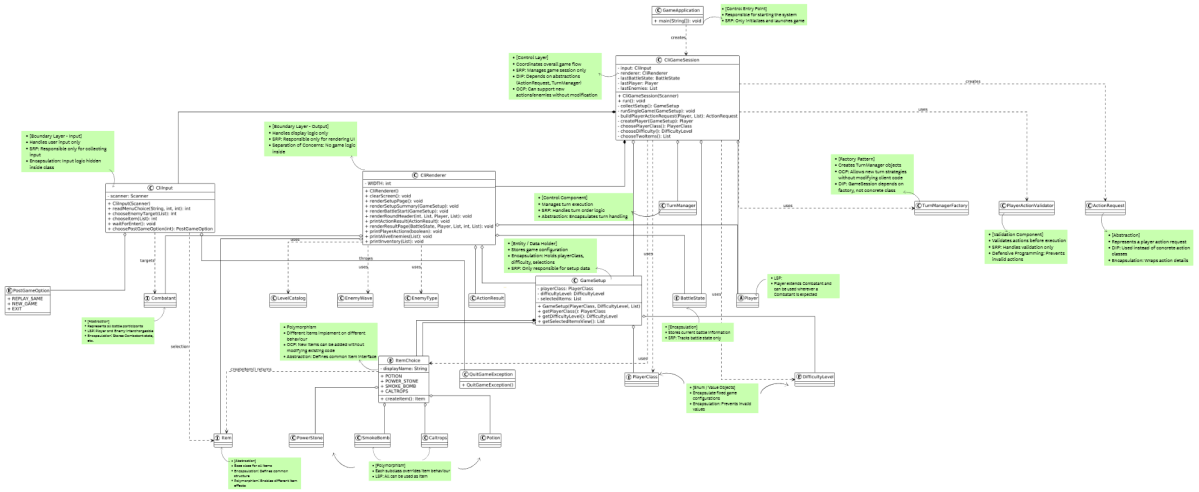
2.2.2: Entity Package UML Class Diagram



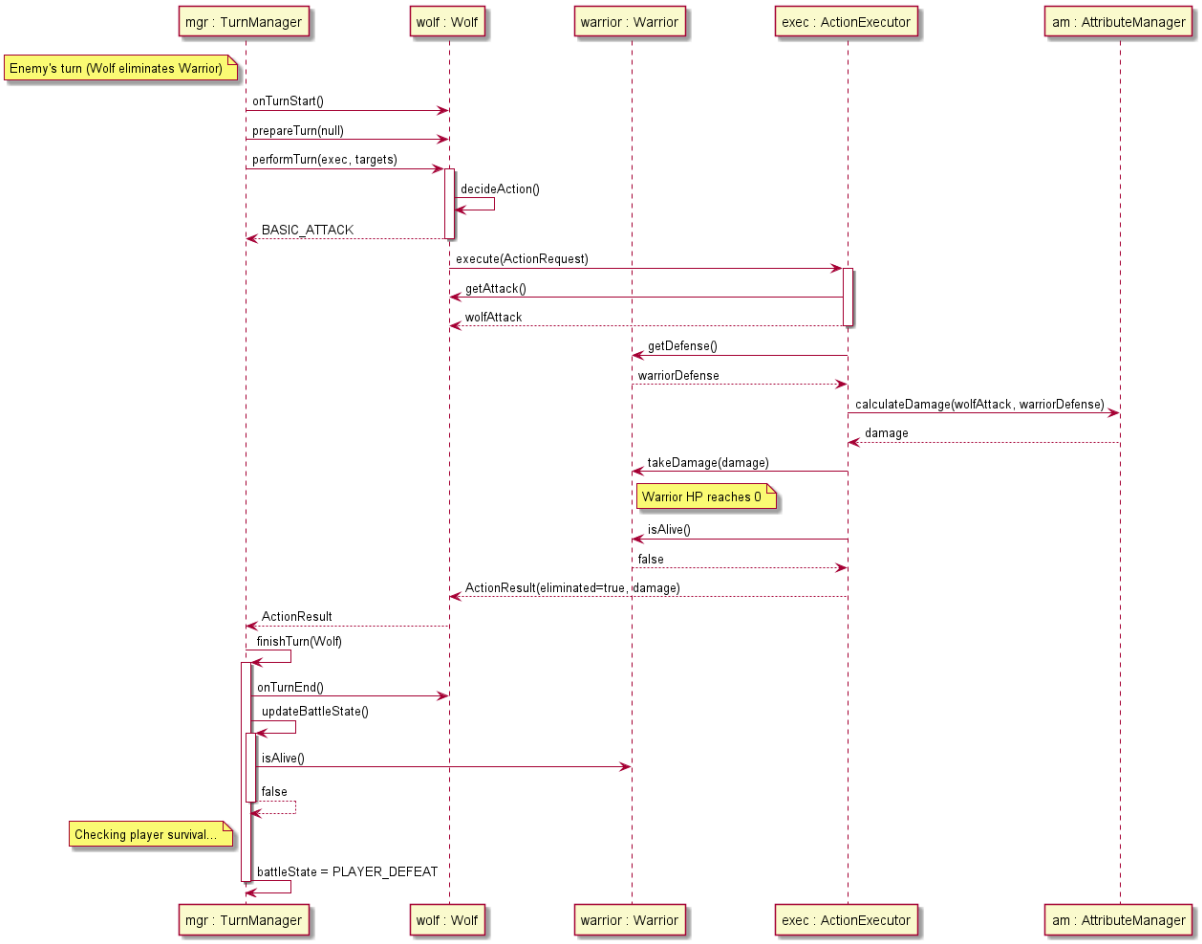
2.2.3: Control Package UML Class Diagram



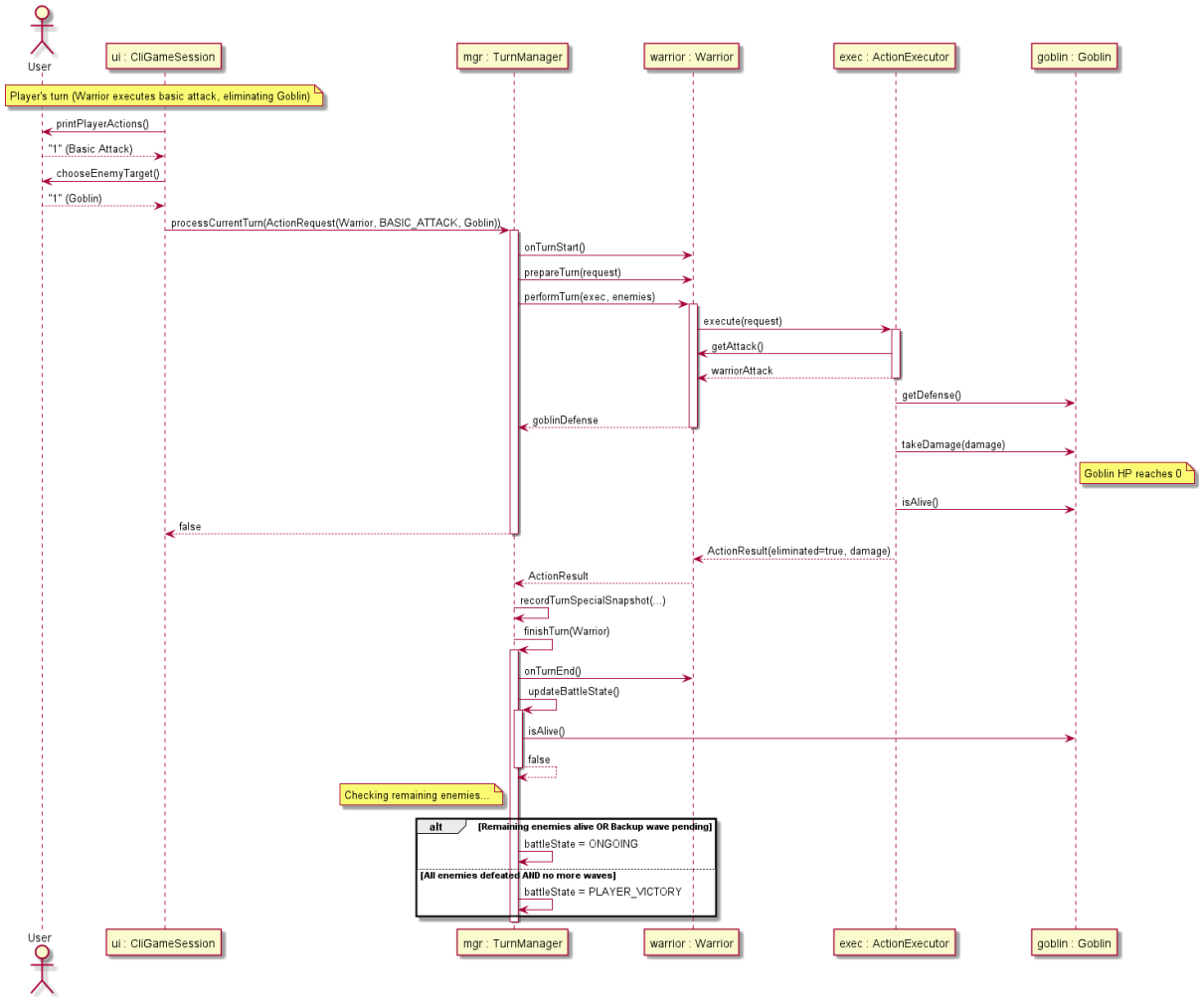
2.2.4: Boundary Package UML Class Diagram



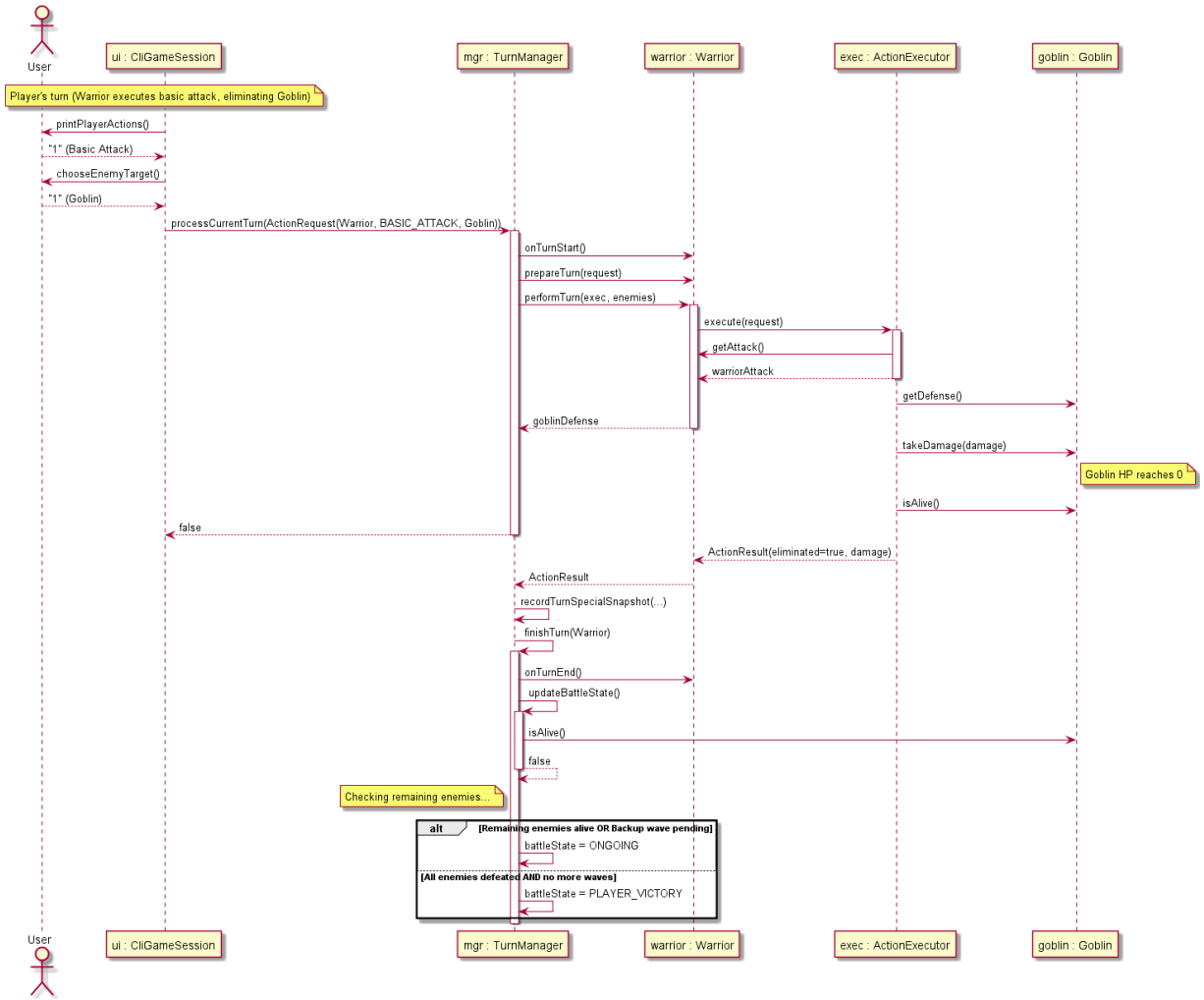
2.2.5: UML sequence diagram for “Wolf uses Basic Attack and eliminate Warrior”



2.2.6: UML sequence diagram for “Warrior does Shield Bash on Goblin, stunning it”



2.2.7: UML sequence diagram for “Warrior executes basic attack, eliminating Goblin”



3. INDIVIDUAL DECLARATIONS

Huy Declaration:

SC2002 Group Assignment AY25/26 Semester 2

Appendix C:

Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to Assignment/Project (to be submitted individually even for group projects)

I Trinh Hoang Huy (student name), hoanghuy001 @e.ntu.edu.sg (NTU email) honestly and sincerely make the following declaration in relation to the following course submission:

1. Name of course: SC2002 Object Oriented Programming and Design

2. Course Code: SC2002

3. Instructor: Dr Li Fang

4. Title of Assignment/Project Submission: Turn-Based Combat Arena

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):

i. Used GAI as permitted to assist in generating key ideas only. ☐

ii. Used GAI as permitted to assist in generating a first text only. ☐

And/or

iii. Used GAI to refine syntax and grammar for correct language submission only.

☐

Or

iv. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project.

☒

I also declare that I have :

a. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that

SC2002 Group Assignment AY25/26 Semester 2

b. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,

c. Apart from the foregoing notices, the submission is wholly my own work.

Trinh Hoang Huy April 17th 2026 Student Name & Signature Date

(Optional) Sample Reflection Questions:

In 150 words or less, briefly state up to 5 key ideas that you formulated that the GAI tools alone could not have generated.

Yuehan Declaration:

SC2002 Group Assignment AY25/26 Semester 2

Appendix C:

Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to Assignment/Project (to be submitted individually even for group projects)

I Wang Yuehan (student name), YWANG243 @e.ntu.edu.sg (NTU email) honestly and sincerely make the following declaration in relation to the following course submission:

1. Name of course: Object Oriented Des & Prog

2. Course Code: SC2002

3. Instructor: Shen Zhiqi

4. Title of Assignment/Project Submission: Turn-based Combat Assignment

In relation to the foregoing, I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):

i. Used GAI as permitted to assist in generating key ideas only.

☐

ii. Used GAI as permitted to assist in generating a first text only.

☐

And/or iii. Used GAI to refine syntax and grammar for correct language submission only.

☐

Or

iv. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project.

☑

I also declare that I have :

a. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that

SC2002 Group Assignment AY25/26 Semester 2

b. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,

c. Apart from the foregoing notices, the submission is wholly my own work.

Wang Yuehan 16/04/2026 Student Name & Signature Date

(Optional) Sample Reflection Questions:

In 150 words or less, briefly state up to 5 key ideas that you formulated that the GAI tools alone could not have generated.

Mahaa Declaration

SC2002 Group Assignment AY25/26 Semester 2

Appendix C:

Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to Assignment/Project (to be submitted individually even for group projects)

I _____Wang Yuehan_____ (student name), _____YWANG243_____@e.ntu.edu.sg (NTU email) honestly and sincerely make the following declaration in relation to the following course submission:

1. Name of course: Object Oriented Des & Prog

2. Course Code: SC2002

3. Instructor: Shen Zhiqi

4. Title of Assignment/Project Submission: Turn-based Combat Assignment

In relation to the foregoing, I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):

i. Used GAI as permitted to assist in generating key ideas only.

☐

ii. Used GAI as permitted to assist in generating a first text only.

☐

And/or iii. Used GAI to refine syntax and grammar for correct language

submission only.

☐

Or

iv. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project.

✓

I also declare that I have :

a. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that

SC2002 Group Assignment AY25/26 Semester 2

b. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,

c. Apart from the foregoing notices, the submission is wholly my own work.

_____

Wang Yuehan 16/04/2026 Student Name & Signature Date

(Optional) Sample Reflection Questions:

In 150 words or less, briefly state up to 5 key ideas that you formulated that the GAI tools alone could not have generated.

Yifan Declaration:

SC2002 Group Assignment AY25/26 Semester 2

Appendix C:

Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to Assignment/Project (to be submitted individually even for group projects)

I _____Zhou Yifan_____ (student name), _____Zhou0633_____@e.ntu.edu.sg (NTU email) honestly and sincerely make the following declaration in relation to the following course submission:

1. Name of course: Object Oriented Des & Prog

2. Course Code: SC2002

3. Instructor: Dr Lifang

4. Title of Assignment/Project Submission: Turn-based Combat Assignment

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):

i. Used GAI as permitted to assist in generating key ideas only.

☐

ii. Used GAI as permitted to assist in generating a first text only.

☐

And/or iii. Used GAI to refine syntax and grammar for correct language

submission only.

☐

Or

iv. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project.

✓

I also declare that I have :

a. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that

SC2002 Group Assignment AY25/26 Semester 2

b. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,

c. Apart from the foregoing notices, the submission is wholly my own work.

_____

_____15/4/2026_____ Student Name & Signature Date

(Optional) Sample Reflection Questions:

In 150 words or less, briefly state up to 5 key ideas that you formulated that the GAI tools alone could not have generated.

Subasri Declaration:

Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to Assignment/Project (to be submitted individually even for group projects)

I, **SUBASRI VAITHIYANATHAN** _____

(student name), _____**SUBASRI002**_____@e.ntu.edu.sg (NTU email) honestly and sincerely make the following declaration in relation to the following course submission:

1. Name of course: **Object Oriented Design and Programming**

2. Course Code: **SC2002**

3. Instructor: **Dr. Li Fang**

4. Title of Assignment/Project Submission: **SC2002 Turn-Based Combat Assignment**

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):

i. Used GAI as permitted to assist in generating key ideas only. ☐ ii.

Used GAI as permitted to assist in generating a first text only. ☐

And/or iii. Used GAI to refine syntax and grammar for correct language submission only.

☐

Or

iv. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project.

☑

I also declare that I have :

a. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that

b. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,

c. Apart from the foregoing notices, the submission is wholly my own work.

18/04/26

SUBASRI VAITHIYANATHAN_____ Student Name & Signature Date

(Optional) Sample Reflection Questions:

In 150 words or less, briefly state up to 5 key ideas that you formulated that the GAI tools alone could not have generated.